

re
shape:

change the shape of an array (without changing its data)

10 20 30 40 50 60

10 20 30

40 50 60

Syntax:

array.reshape (rows, columns)

import numpy as np

arr = np.array ([10 20 30 40 50 60])

print (new-arr)

output:
[10 20 30
40 50 60]

arr.reshape(4, 2) X (not execute) it show error

Automatic reshaping

Instead of calculating rows numpy

can calculate automatically using

```
arr = np.arange(1, 13)
```

```
new_arr = arr.reshape(3, -1)
```

```
print(new_arr)
```

Output:

[1 2 3 4

5 6 7 8

9 10 11 12]

Flatten()

[1 2 3

4 5 6]

[1 2 3 4 5 6]

→ M1 algorithm accept

```
arr = np.array([1 2 3 4 5 6])
```

```
print(arr.flatten())
```

Output

[1 2 3 4 5 6]

Copy:

Original array not changed

```
np.array([10, 20, 30])
```

```
new_arr = arr.copy()
```

```
new_arr[0] = 100
```

```
print(arr)
```

```
print(new_arr)
```

Output

[10 20 30]

[100 20 30]

Mathematical operation in array:

Addition

```
arr = np.array([10, 20, 30])
```

```
print(arr + 5)
```

Output

```
[15 25 35]
```

Sub

```
(arr - 5)
```

power

```
(arr ** 5)
```

Mul

```
(arr * 5)
```

One array and another array operation

```
a = np.array([1, 2, 3])
```

```
b = np.array([10, 20, 30])
```

```
print(a + b)
```

Output:

```
[11 22 33]
```

Broadcasting (X)

Broadcasting Allow numpy array of different shapes (sizes) to perform arithmetic operations without explicitly making them to same size

```
arr = np.array([10, 20, 30])
```

```
print(arr + 5)
```

sgn

array + value

array + array =

array = value

array * array =

Building Function

Square root

`print(np.sqrt(arr))` $\Rightarrow$ square root Universal Buildin function

`print(np.exp(1, 2, 3))` $\Rightarrow$ exponential.

`print(np.log(1, 2, 4))` $\Rightarrow$ logarithm

`print(np.sgnsin(90))` $\Rightarrow$ trigonometry functions
<sub>cos
tan</sub>

Statistical Function

Sum: `print(arr.sum())`

Mean: `print(arr.mean())` $\Rightarrow$ average

Median: `print(arr.median())` $\Rightarrow$ center

Maximum: `print(arr.max())`

Minimum: `print(arr.min())`

Standard deviation: `print(arr.std())` $\Rightarrow$ How spread in data
Use in machine learning & statistics

matrix multipl } $A \times b$

q.m.

variables